Phuthiphong Wongchantib 660610840

หุ่นยนต์ที่เลี้ยวได้ทางเดียว (ทวนเข็มนาฬิกา) — อัลกอริทึม Detour Steer

วิ่งไปก่อน เลี้ยวให้น้อยลง แล้วค่อยเข้าเป้า

ตัวหุ่น มีมอเตอร์ DC ตัวเดียว หมุนทางหนึ่งเป็นการเลี้ยวชุดล้อ ได้ทวนเข็มนาฬิกาเท่านั้น (มีคลัตช์ทางเดียว) หมุนอีกทางเป็นการวิ่งไปข้างหน้า หุ่นจึงเลี้ยวกับวิ่งพร้อมกันไม่ได้ วิธีเดิมคือเลี้ยวตรงไปหาเป้าก่อนแล้วค่อยวิ่ง ถ้าเป้าอยู่ทางขวานิดเดียว (φ ใกล้ 360°) ล้อต้องหมุนเกือบรอบ **แนวคิด** เส้นตรงสองช่วงที่ทิศต่างกันเกิน 180° ไปถึงได้ทุกทิศ หุ่นจึงเลือกได้ว่าจะวิ่งตรงไปก่อน a เมตร เลี้ยวแค่ β^* ($180^\circ < \beta^* < \varphi$) แล้ววิ่งอีก b เมตร ถึงเป้า มุม β^* ที่ดีที่สุดมีสูตรปิดคำนวณได้ทันที มุมทุกมุมวัดทวนเข็มนาฬิกาจากแกน $+x$; ω = ความเร็วเลี้ยว, v = ความเร็ววิ่ง, t_s = เวลารอล้อนิ่ง, t_b = เวลาหยุดเปลี่ยนจากวิ่งเป็นเลี้ยว, tol = ค่าคลาดเคลื่อนที่ยอมได้, r_{goal} = รัศมีถือว่าถึงเป้า **บนหุ่นจริง** (`mor_luam`, ESP32) ชุดล้อเลี้ยวทวนเข็มเมื่อมองจากด้านบน ตรงกับระบบมุมในหน้านี้ (ชื่อฟังก์ชัน cwErrorDeg ในเฟิร์มแวร์เป็นชื่อเก่า แต่ค่าที่ได้คือมุมทวนเข็ม) อัลกอริทึม 1.1–1.5 คือการวางแผน ส่วน 1.6–1.7 คือวิธีที่หุ่นเลี้ยวและวิ่งแต่ละช่วงจริง

อัลกอริทึม 1.1: ฟังก์ชันช่วยเรื่องมุม (ทวนเข็มนาฬิกา)

Procedure : Bearing(Δx, Δy) // มุมทวนเข็มของทิศ $(\Delta x, \Delta y)$
  return $\operatorname{atan2}(\Delta y, \Delta x) \bmod 2\pi$;
end

Procedure : SteeringError(θ, h) // ล้อต้องเลี้ยวทวนเข็มอีกเท่าไร: $0 \ldots 2\pi$
  return $(\theta - h) \bmod 2\pi$;
end

Procedure : InTolerance(e) // ใกล้พอ ไม่ว่าจะขาดหรือเกิน
  return $e \le tol$ **or** $e \ge 2\pi - tol$; // เลยเกินเป้านิดเดียวจะอ่านได้ $e \approx 2\pi$
end

อัลกอริทึม 1.2: มุมอ้อมที่ดีที่สุดด้วยสูตรปิด, $O(1)$

Procedure : OptimalDetourAngle(φ, d) // φ: มุมทวนเข็มไปหาเป้า, d: ระยะ
  if $\varphi \le \pi$ **then return None**; // ทฤษฎีบท 1: เลี้ยวตรงไปหาเป้าดีที่สุด
  $k \leftarrow (d\,\omega / v)\,|\sin\varphi|$; // ตัวเลขตัวเดียวตัดสินทุกอย่าง
  if $k \ge 2$ **then return None**; // ทฤษฎีบท 3: เวลาลดลงเรื่อย ๆ จนถึง $\beta = \varphi$
  $\beta^* \leftarrow 2\pi - \arccos(k-1)$; // รากเดียวของ $dT/d\beta = 0$ ในช่วง $(\pi, 2\pi)$
  if $\beta^* \ge \varphi$ **then return None**; // มุมที่ดีที่สุดไม่ใช่การอ้อม
  return β^*;
end

อัลกอริทึม 1.3: วางแผนหนึ่งช่วง: ไปตรงหรืออ้อม, $O(1)$

Procedure : PlanLeg(φ, d) // คืนค่า (แบบ, a, β, b, เวลา)
  $T_D \leftarrow \varphi/\omega + t_s + d/v$; // วิธีเดิม: เลี้ยว φ แล้ววิ่ง d
  $\beta^* \leftarrow$ OptimalDetourAngle(φ, d);
  if $\beta^* =$ **None then return** (DIRECT, 0, φ, d, T_D);
  $a \leftarrow d\sin(\beta^* - \varphi)/\sin\beta^*$; // ช่วงที่ 1: ทิศเดิม ไม่เลี้ยว
  $b \leftarrow d\sin\varphi/\sin\beta^*$; // ช่วงที่ 2: ทิศ β^* จบที่เป้า
  $T_R \leftarrow a/v + t_b + \beta^*/\omega + t_s + b/v$; // หยุดเพิ่มหนึ่งครั้งระหว่างสองช่วง
  if $T_R < T_D$ **then return** (DETOUR, a, β^*, b, T_R); // อ้อมเฉพาะเมื่อเร็วกว่าจริง
  return (DIRECT, 0, φ, d, T_D);
end

อัลกอริทึม 1.4: Detour Steer แบบวงปิดไปหาเป้าหนึ่งจุด (วางแผนใหม่หลังขยับทุกครั้ง)

Procedure : DetourSteer(p, h, g) // p: ตำแหน่ง, h: ทิศของล้อ, g: เป้า
  while $\|g - p\| > r_{goal}$ **do**
    $d \leftarrow \|g - p\|$; $e \leftarrow$ SteeringError(Bearing($g - p$), h);
    if InTolerance(e) **then** // เล็งตรงอยู่แล้ว: วิ่งไปหาเป้า
      DriveReaim(d); // อัลกอริทึม 1.7
    else
      $plan \leftarrow$ PlanLeg(e, d);
      if $plan.kind =$ DETOUR **then**
        DriveReaim($plan.a$); // ช่วงที่ 1: วิ่งตรงไปก่อน ไม่เลี้ยว
        Stop(); // t_b: สลับมอเตอร์จากวิ่งเป็นเลี้ยว
      else
        SteerOneWay(Bearing($g - p$)); // อัลกอริทึม 1.6: ตั้งใจหยุดก่อนถึงมุมนิดหนึ่ง
      end
    end
    $p \leftarrow$ MeasurePosition(); $h \leftarrow$ MeasureHeading(); // มุมล้อ AS5600 + ทิศตัวหุ่น IMU; T4
    // ขาด ε: $e = \varepsilon$ เลี้ยวเพิ่มนิดเดียว เกิน ε: $e = 2\pi - \varepsilon$ ให้ PlanLeg เลือกอ้อม
  end
  return (p, h);
end

อัลกอริทึม 1.5: วิ่งตามเส้นทางทั้งหมดด้วย Detour Steer

Procedure : FollowRouteDetour(W) // ฟังก์ชันหลัก: ไปตามเป้า W ทีละจุดตามลำดับ
  $p \leftarrow (0, 0)$; $h \leftarrow 0$; // ท่าเริ่มต้น
  foreach $g \in W$ **do**
    $(p, h) \leftarrow$ DetourSteer(p, h, g);
  end
  แสดง เวลารวม; // เป้า 1 m ที่ 350°: a=1.03 m, β^*=254°, b=0.18 m, 9.34 s (วิธีเดิม 9.88 s)
end

อัลกอริทึม 1.6: เลี้ยวทางเดียว: ตั้งใจหยุดก่อนถึงมุมนิดหนึ่ง (บนหุ่น ทุก 10 ms)

Procedure : SteerOneWay(θ) // θ: ทิศที่ต้องการ
  while true **do** // ทุกรอบควบคุม 10 ms
    $h \leftarrow$ MeasureHeading(); // ล้อ (AS5600) + ตัวหุ่น (IMU): ตัวหุ่นบิดตามตอนเลี้ยว
    $e \leftarrow$ SteeringError(θ, h); $r \leftarrow$ ความเร็วเลี้ยวในช่วง 50 ms;
    if $e < \pi$ **and** $e \le e_{land} + r\,c$ **then break**; // ตัดไฟก่อน: ล้อไหลต่อไปหยุดที่ขาด $\approx e_{land}$
    if InTolerance(e) **and** ($e > \pi$ **or** $r < r_{min}$) **then break**; // เกินแล้ว หรือล้อนิ่ง: ดันจากนิ่งจะกระโดดเกิน
    เปิดมอเตอร์เลี้ยว;
  end
  ปิดมอเตอร์เลี้ยว; รอจนล้อหยุดและตัวหุ่นหมุน $< 5^\circ$/s (นานสุด 0.4 s);
  $c \leftarrow (1-\lambda)\,c + \lambda\,(\text{มุมที่ไหลต่อ}/r_{cut})$; // เรียนรู้เวลาไหลต่อ c
end

อัลกอริทึม 1.7: วิ่งพร้อมเล็งใหม่ก่อนถึง (มอเตอร์ตัวเดียว เลี้ยวขณะวิ่งไม่ได้)

Procedure : DriveReaim(ℓ) // ℓ: ความยาวช่วงในทิศปัจจุบัน h
  $q \leftarrow p + \ell\,(\cos h, \sin h)$; // จุดปลายของช่วงนี้
  while true **do** // ทุกรอบควบคุม 10 ms
    $(f, s) \leftarrow q - p$ มองจากล้อ; // f ระยะข้างหน้า, s ระยะด้านข้าง (+: ฝั่งที่เลี้ยว)
    if $\|q - p\| \le r_{goal}$ **or** $f \le 0$ **then break**; // ถึงแล้ว หรือมาเสมอจุดแล้ว
    if $s > r_{goal}$ **and** $f \le \max(m, R\,s)$ **then break**; // จากนั้นเล็งใหม่แค่ $\arctan(1/R)$
    เปิดมอเตอร์วิ่ง;
  end
  ปิดมอเตอร์วิ่ง; $p \leftarrow$ MeasurePosition(); // อัลกอริทึม 1.4 วางแผนใหม่จากตรงนี้
end

ถ้าเป้าอยู่ที่มุมทวนเข็ม $\varphi > \pi$ ระยะ d การอ้อมที่จบด้วยทิศ β ใช้เวลา

$$T(\beta) = \frac{\beta}{\omega} + t_s + t_b + \frac{d}{v}\,g(\beta), \qquad g(\beta) = \frac{\sin(\beta-\varphi)+\sin\varphi}{\sin\beta}, \qquad g'(\beta) = \frac{\sin\varphi}{1+\cos\beta}.$$

T1. ถ้า $\varphi \le 180^\circ$ การเลี้ยวตรงไปหาเป้าดีที่สุด ทิศสองทิศที่ห่างกันไม่ถึง 180° ไปได้แค่ทิศที่อยู่ระหว่างกัน ทุกเส้นทางจึงต้องเลี้ยวอย่างน้อย φ และต้องวิ่งอย่างน้อย d เส้นทางตรงทำได้ทั้งสองอย่างพอดี

T2. ถ้า $\varphi > 180^\circ$ เส้นทางที่ดีที่สุดมีไม่เกิน 2 ช่วง ทิศ 0 และ β การเขียนเวกเตอร์ไปเป้าให้สิ้นที่สุดด้วยทิศในช่วง $[0, \beta]$ ใช้สองทิศที่อยู่คนละข้างของ φ (คำตอบที่ดีที่สุดของ linear program 2 แถว มีตัวแปรไม่เป็นศูนย์แค่ 2 ตัว)

T3. บนช่วง $(\pi, 2\pi)$ อนุพันธ์ $T'(\beta) = 1/\omega - (d/v)|\sin\varphi|/(1+\cos\beta)$ เพิ่มขึ้นตลอด T จึงมีจุดต่ำสุดจุดเดียว: $\cos\beta^* = k-1$ เมื่อ $k = (d\omega/v)|\sin\varphi|$ ถ้า $k \ge 2$ เวลา T ลดลงเรื่อย ๆ จนถึง $\beta = \varphi$ ซึ่งก็คือการไปตรง

T4. หลังวิ่งช่วงที่ 1 แล้ว PlanLeg(β^*, b) จะได้ "เลี้ยว β^* แล้ววิ่ง b" เสมอ การอ้อมแบบอื่นจากจุดนั้นเท่ากับช่วงที่ 1 ยาวขึ้นบวกการหยุดเพิ่มอีก t_b ซึ่ง T3 ชนะไปแล้ว จึงวางแผนใหม่หลังขยับทุกครั้ง (วงปิด) ได้โดยแผนไม่เปลี่ยน

O. PlanLeg() ใช้เวลาและหน่วยความจำ $O(1)$: 0.54 μs ต่อครั้งใน Python เทียบกับ 505 μs ของการค้นหา β 3600 จุด (เร็วกว่า $944\times$) T1–T4 ตรวจแบบ brute force ใน `detour_steer.py` ด้วย: สูตรปิดเทียบตาราง 20 000 จุด, เทียบทุกเส้นทาง 2 ช่วงบนตารางทุก 2° และวางแผนซ้ำ 2000 ครั้ง ผ่านทั้งหมด

ความคลาดเคลื่อนตอนหยุดที่ไม่รู้ล่วงหน้า ไม่มีใครรู้ก่อนว่าล้อจะหยุดเกินหรือขาดไปนิดหนึ่ง หุ่นอ่านมุมล้อจริง (AS5600) และทิศตัวหุ่น (IMU) หลังหยุด แล้ว DetourSteer() วางแผนใหม่จากตรงนั้น PlanLeg() ไม่ใช้ค่าความคลาดเคลื่อนเลย ถ้าอยู่ใน $\pm tol$ หุ่นวิ่งไปตามทิศจริงของมัน และถ้าพลาดเป้าเกิน r_{goal} ก็วางแผนใหม่

ทำไมตั้งใจหยุดก่อนถึงมุม (อัลกอริทึม 1.6) ล้อที่เลี้ยวได้ทางเดียว ถ้าเกินมุมไปมากกว่า tol ต้องหมุนอีกเกือบรอบ แต่ถ้าขาดก็เลี้ยวเพิ่มนิดเดียว หุ่นจึงตัดไฟมอเตอร์ก่อนแล้วให้ล้อไหลต่อไปหยุดที่ $e_{land} = 2^\circ$ ก่อนถึงมุม (c คือเวลาไหลต่อ เรียนรู้ใหม่ทุกครั้งที่เลี้ยว) ล้อที่นิ่งอยู่ในช่วงยอมรับแล้วจะไม่ถูกดันอีก เพราะดันจากนิ่งมอเตอร์ต้องเร่งกำลังขึ้น แล้วล้อกระโดดเกิน (วัดได้ 9°) การหยุดขาดยังทำให้เป้าอยู่ฝั่งที่เลี้ยวของเส้นที่หุ่นวิ่งเสมอ DriveReaim() จึงแก้ได้ด้วยการเลี้ยวนิดเดียว ($\arctan(1/R) = 18^\circ$ เมื่อ $R = 3$, $m = 2$ cm) แทนการเลี้ยว 90° หลังวิ่งเลยเป้า

ผลจากซิม (เป้าสุ่ม 5000 จุด, $d \in [0.3, 2]$ m, $v = 0.25$ m/s, $t_b = 0.2$ s, $\omega = 60^\circ$/s ใช้เป้าและความคลาดเคลื่อนชุดเดียวกันทั้งสองวิธี, เวลาเฉลี่ยต่อเป้า):

ล้อหยุด	bias	sd	วิธีเดิม (s)	Detour (s)	ประหยัด
ตรงมุมพอดี	0°	0°	7.56	7.53	0.4 %
สุ่มได้ทั้งสองทาง	0°	2°	8.42	8.23	2.3 %
สุ่มได้ทั้งสองทาง	0°	3°	9.43	8.78	6.9 %
ส่วนใหญ่ขาด	-2°	2°	8.08	8.03	0.7 %
ส่วนใหญ่เกิน	$+2^\circ$	2°	10.74	9.36	12.8 %
ส่วนใหญ่เกิน	$+2^\circ$	3°	11.87	9.70	18.3 %

ทุกกรณี Detour Steer ไม่ช้ากว่าวิธีเดิมเลย (0 % ของเป้า) ถ้าล้อหยุดตรงพอดี ค่าเฉลี่ยดีขึ้นไม่มาก แต่เป้าที่อยู่ทางขวานิดเดียวเร็วขึ้นได้ถึง 28 % ยิ่งล้อหยุดเกินบ่อย ยิ่งได้ประโยชน์ เพราะการเกินไม่ต้องเสียเวลาหมุนอีกรอบแล้ว เลี้ยวซ้ายลงก็ได้ประโยชน์มากขึ้น (ที่ 30°/s, sd 2°: 4.8 %)

ผลบนหุ่นจริง (`mor_luam`: $v = 0.03$ m/s, $\omega = 35^\circ$/s ซึ่งเป็นความเร็วเลี้ยวที่วัดได้, $t_b = 0.2$ s, $r_{goal} = 2.5$ mm) งานเดียวกันทั้งสองวิธี: ก่อนเริ่มจับเวลา ล้อถูกหมุนไปทิศเริ่มต้นและนิ่งแล้ว จากนั้นวางเป้าห่าง $d = 0.30$ m ที่มุม $\varphi = 354^\circ$ จากทิศที่ล้อชี้อยู่จริง หุ่นจับเวลาเองทุกรอบและวิ่งกลับจุดเริ่ม (ไม่จับเวลาขากลับ) ทำ 3 รอบ ลำดับ เดิม–Detour, Detour–เดิม, เดิม–Detour:

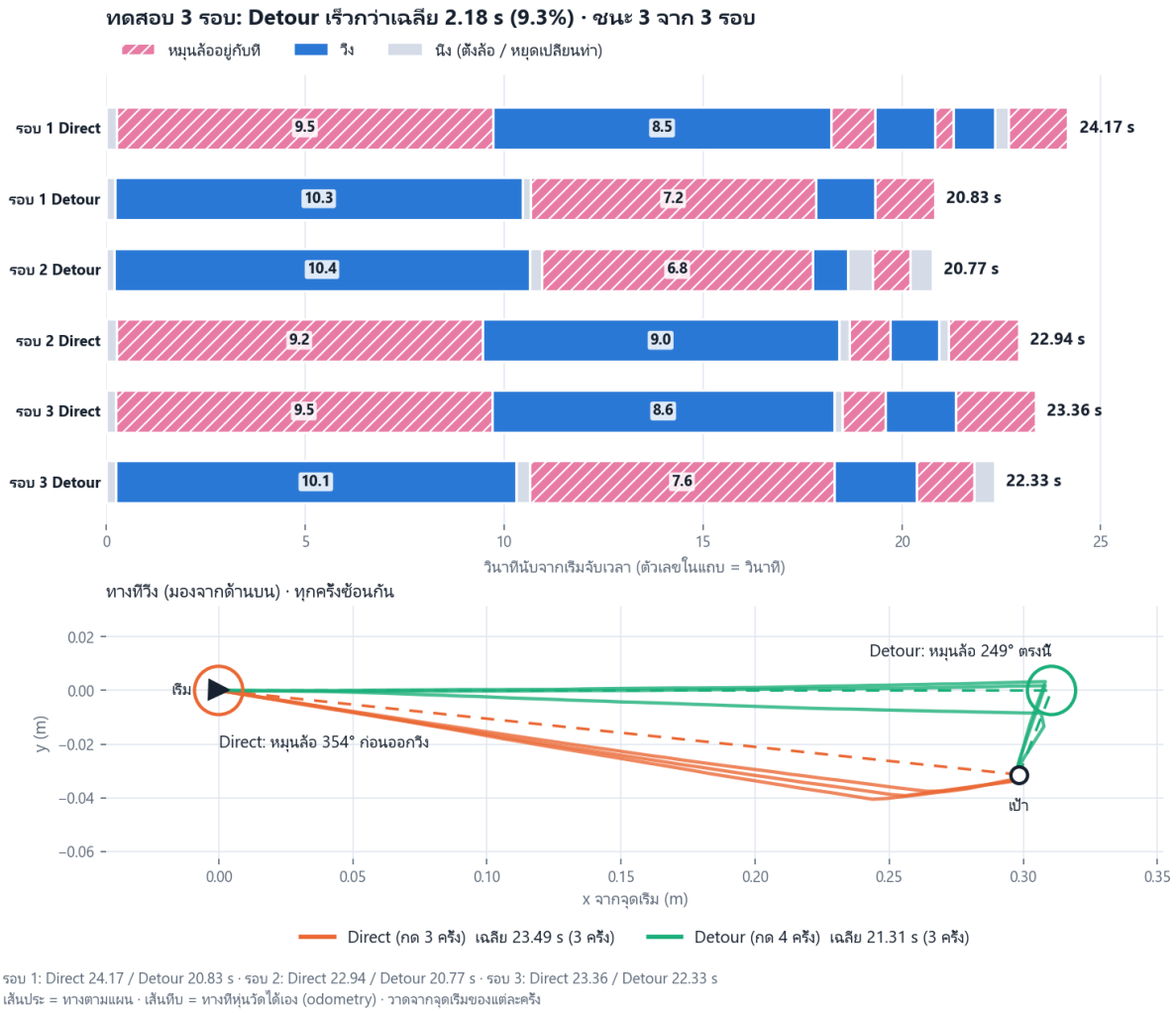
รอบ	วิธีเดิม (s)	Detour Steer (s)	ประหยัด (s)
1	24.17	20.83	3.34
2	22.94	20.77	2.17
3	23.36	22.33	1.03
เฉลี่ย	23.49	21.31	2.18 (9.3 %)

Detour Steer เร็วกว่าทั้ง 3 รอบ เลี้ยวน้อยลง 3.8 s (8.5 เทียบ 12.4 s) และวิ่งนานขึ้น 1.2 s (11.8 เทียบ 10.6 s) แผนทำนายไว้ 18.8 เทียบ 20.2 s แต่บนพื้นจริงต่างกันมากกว่า เพราะวิธีเดิมที่หมุนเกือบรอบต้องแก้ทิศตอนท้ายมากกว่าด้วย ทั้ง 6 รอบหยุดห่างเป้าไม่เกิน 2.5 mm

งานที่ใกล้เคียงที่สุด และความต่าง กลไกมอเตอร์ตัวเดียวทั้งเลี้ยวและวิ่ง (US 8,424,627 B2, 2013; Sekine & Mizuuchi, ICINCO 2022) ไม่มีตัววางแผน รถแบบ Dubins ที่เลี้ยวได้ทางเดียว (Bakolas & Tsiotras, JOTA 2011; Patsko & Fedotov, 2018–2019) เลี้ยวขณะ

เคลื่อนที่ การวางเส้นทางที่คิดต้นทุนการเลี้ยว (angular-metric และ quadratic TSP) ใช้จัดลำดับจุด และการหยุดล่วงหน้าแบบทำนายการไหล (US 8,571,394 B2) แค่หยุดก่อน ยังไม่พบงานใดที่ให้สูตรปิดของการอ้อมแบบนี้ สำหรับหุ่นที่ต้องหยุดก่อนเลี้ยว และเลี้ยวได้แค่ทวนเข็ม ทั้งนี้ “ไม่พบ” ไม่ได้พิสูจน์ว่าไม่มีใครเคยทำ

รูปที่ 1 ผล 3 รอบบนหุ่นจริง จากข้อมูลที่หุ่นบันทึกเอง



รอบ 1: Direct 24.17 / Detour 20.83 s · รอบ 2: Direct 22.94 / Detour 20.77 s · รอบ 3: Direct 23.36 / Detour 22.33 s
เส้นประ = ทางตามแผน · เส้นทึบ = ทางที่หุ่นวัดได้เอง (odometry) · วาดจากจุดเริ่มของแต่ละครั้ง

บน: สิ่งที่แต่ละรอบทำตามเวลา (ลายชมพู = เลี้ยวอยู่กับที่, น้ำเงิน = วิ่ง, เทา = นิ่ง) วิธีเดิมเลี้ยวอยู่กับที่ก่อน ≈ 9.4 s แล้วค่อยวิ่ง ส่วน Detour Steer วิ่งทันทีและเลี้ยวน้อยกว่าในภายหลัง **ล่าง:** เส้นทางที่หุ่นวัดได้ (odometry) ทั้ง 6 รอบจากจุดเริ่มเดียวกัน มองจากด้านบน เส้นประ = แผน (หุ่นเลี้ยวทวนเข็ม ในรูปนี้เป้าจึงอยู่ทางขวา)

ค่าของหุ่นปัจจุบัน และการเปรียบเทียบระยะ 2 เมตร (27 ตุลาคม 2026)

ผล benchmark ด้านบนใช้ความเร็วอ้างอิงทางทฤษฎี 0.25 m/s ไม่ใช่ความเร็วของหุ่นจริง ค่าที่วัดได้ซึ่งใช้วางแผนคือ $v = 0.03$ m/s, $\omega = 35^\circ$/s และรัศมีถึงเป้า 0.0025 m สำหรับการเปรียบเทียบระยะยาว แบ่งระยะขาไปรวม 2 m เป็น 7 ช่วงเท่ากัน ช่วงละ 2/7 m แต่ละช่วงหักไปทางตามเข็ม 6° จากทิศของช่วงก่อน ทั้งสองวิธีไปจุดเดียวกัน และหุ่นยังเลี้ยวทวนเข็มเหมือนเดิม แบบจำลองอุดมคติให้ Direct 137.82 s / 2.00 m และ Detour 110.52 s / 2.17 m โดยอ้อม 4 ครั้ง ค่านี้เป็นค่าทำนายจากแบบจำลอง ไม่ใช่ผลวัดใหม่บนหุ่นจริง ขากลับจุดเริ่ม (ประมาณ 1.96 m) ไม่นับเวลา ทดสอบ 3 คู่ รวม 6 รอบ การบันทึกค่าตั้งค่าไม่ทำให้หุ่นเริ่มวิ่งเอง

ภาคผนวก ก. โค้ดและ Git

โค้ดทั้งหมดของอัลกอริทึมนี้ (ตัววางแผน Python, ซิม, เฟิร์มแวร์บนหุ่น และรายงานนี้) อยู่บน Git:

`https://github.com/Mannaja78520/mor_luam` (branch `main`)

วิดีโอการทำงานของหุ่น:

`https://drive.google.com/drive/folders/11qOM6UoeH2c4nEuM0a0JO6eXTnr0eDud`

คลิปนี้เป็นการทำงานของหุ่นแบบ synchronize ไม่มีคลิปของการเปรียบเทียบ Detour Steer กับวิธีเดิม (Direct) ผลการเปรียบเทียบดูได้จากข้อมูลที่หุ่นบันทึกเองในรูปที่ 1 และตารางผลบนหุ่นจริง

อัลกอริทึมตัดสินใจอยู่ที่ไหน

- `algorithm_sim/detour_steer.py` – ตัววางแผนต้นแบบ ตรวจทฤษฎีบท T1–T4 benchmark และสร้างรูป
- `algorithm_sim/sim_detour_steer.py`, `sim_detour_only.py` – หน้าต่างซิมบนคอม
- `firmware/src/algorithm/DetourSteer.{h,cpp}` – ตัววางแผนเดียวกันในภาษา C++ บน ESP32
- `firmware/src/nav/WaypointRunner.cpp` – สั่งหุ่นวิ่งตามแผนทีละช่วง; `FinalApproach.h` และ `SteerApproach.h` – อัลกอริทึม 1.6 และ 1.7

ดาวน์โหลดโค้ดและรันซิม (ไม่ต้องใช้ Docker)

```
git clone https://github.com/Mannaja78520/mor_luam.git
cd mor_luam/algorithm_sim
pip install matplotlib
python detour_steer.py --no-bench      # theorem checks T1-T4 + figures
python sim_detour_steer.py             # old method vs Detour Steer
python -m unittest test_current_robot
```

Build และอัปโหลดเฟิร์มแวร์ลงหุ่น (แนะนำติดตั้งแบบ Docker)

```
docker\mor_luam.bat fw-build              # Windows, Docker Desktop running
docker\mor_luam.bat fw-flash -Port COM9   # first time, over USB
docker\mor_luam.bat fw-ota -Host <IP>     # later, over Wi-Fi
./docker/mor_luam.sh fw-build             # Ubuntu, same commands
```

วิธีติดตั้งทีละขั้น: `docs/INSTALL_TH.md` ใน repository

ก.1 ตัววางแผนใน Python (algorithm_sim/detour_steer.py)

```python
def optimal_detour_angle(phi: float, d: float, robot: RobotConfig) -> Optional[float]:
    """beta* in (pi, phi), or None when no detour can beat steering straight.

    phi: counter-clockwise angle from the wheel to the goal (0 .. 2*pi)
    d  : distance to the goal (m)
    """
    if phi <= math.pi:                      # T1: straight at the goal is optimal
        return None
    w = math.radians(robot.steer_speed_dps)
    k = (d * w / robot.drive_speed_mps) * abs(math.sin(phi))
    if k >= 2.0:                            # T3: time falls all the way to phi
        return None
    beta = TAU - math.acos(k - 1.0)         # T3: the one root of dT/dbeta = 0
    return beta if beta < phi else None


def detour_legs(phi: float, d: float, beta: float) -> Tuple[float, float]:
    """Lengths of leg 1 (heading 0) and leg 2 (heading beta) that end at the goal."""
    s = math.sin(beta)
    return d * math.sin(beta - phi) / s, d * math.sin(phi) / s


# ====================================================================
# Algorithm 1.3: plan one leg (direct or detour)
# ====================================================================


@dataclass(frozen=True)
class Plan:
    kind: str            # "direct" or "detour"
    a: float             # leg 1, driven first with NO steer (0 for direct)
    beta: float          # the one counter-clockwise steer
    b: float             # leg 2, to the goal
    time_s: float


def direct_time(phi: float, d: float, robot: RobotConfig) -> float:
    """The current method: steer phi counter-clockwise, then drive d."""
    w = math.radians(robot.steer_speed_dps)
    steer = phi / w + robot.settle_s if phi > 1e-12 else 0.0
    return steer + d / robot.drive_speed_mps


def detour_time(a: float, beta: float, b: float, robot: RobotConfig) -> float:
    w = math.radians(robot.steer_speed_dps)
    v = robot.drive_speed_mps
    return a / v + robot.stop_s + beta / w + robot.settle_s + b / v


def plan_leg(phi: float, d: float, robot: RobotConfig) -> Plan:
    """O(1). The faster of 'steer phi, drive d' and the best detour."""
    direct = Plan("direct", 0.0, phi, d, direct_time(phi, d, robot))
    beta = optimal_detour_angle(phi, d, robot)
    if beta is None:
        return direct
    a, b = detour_legs(phi, d, beta)
    t = detour_time(a, beta, b, robot)
    # The detour adds one stop (drive -> steer), so it must really be faster.
    return Plan("detour", a, beta, b, t) if t < direct.time_s else direct
```

ก.2 ตัววางแผนบนหุ่น (firmware/src/algorithm/DetourSteer.h)

```cpp
#pragma once
// Detour Steer - the algorithm of the 271401 homework
// (E:\271401\271401_pee_aut_homework\algorithm\DETOUR_STEERING\algorithm.pdf,
//  reference implementation: aut_HW_04_detour_steering\detour_steer.py).
//
```

```
// A wheel that steers one way only pays almost a full turn for a goal just
// "behind" its rotation. Two straight legs whose headings differ by more than
// 180 deg reach any direction, so instead of steering phi the robot may:
//   1. drive a metres along its current heading (no steer),
//   2. steer only beta* (180 < beta* < phi),
//   3. drive b metres to the goal.
//
//   k     = (d * omega / v) * |sin phi|
//   beta* = 360 - acos(k - 1)            only if phi > 180, k < 2, beta* < phi
//   a     = d * sin(beta* - phi) / sin(beta*)
//   b     = d * sin(phi) / sin(beta*)
//
// and the detour is taken only when it is really faster than steering
// straight (it costs one extra stop t_b). Theorems T1-T4 in the PDF: optimal,
// at most two legs, the closed form is the minimum, and planning again after
// leg 1 gives "steer beta*, drive b" - so the caller simply plans again after
// every move, and a wheel that stops PAST its angle is recovered the same way.
#include "algorithm/LegPlanner.h"

class DetourSteer : public LegPlanner {
public:
    const char* name() const override { return "detour"; }
    LegPlan plan(float phiDeg, float d, const RobotParams& p) const override;

    // Algorithm 1.2: beta* in degrees, or a negative value when no detour can
    // beat steering straight. Exposed for tests.
    static float optimalDetourAngleDeg(float phiDeg, float d, const RobotParams& p, float* kOut = nullptr);
};
```

ก.3 ตัววางแผนบนหุ่น (firmware/src/algorithm/DetourSteer.cpp)

```
#include "algorithm/DetourSteer.h"
#include <math.h>
#include "algorithm/DirectPlanner.h"

namespace {
const float PI_F = 3.14159265358979f;
const float TAU_F = 2.0f * PI_F;
inline float rad(float deg) { return deg * PI_F / 180.0f; }
inline float deg(float r) { return r * 180.0f / PI_F; }
}  // namespace

float DetourSteer::optimalDetourAngleDeg(float phiDeg, float d, const RobotParams& p, float* kOut) {
    const float phi = rad(phiDeg);
    if (kOut) *kOut = 0.0f;
    if (phi <= PI_F) return -1.0f;                          // T1: straight at the goal is optimal
    const float w = rad(p.steerDps);
    const float k = (d * w / p.driveMps) * fabsf(sinf(phi));
    if (kOut) *kOut = k;
    if (k >= 2.0f) return -1.0f;                            // T3: time keeps falling up to beta = phi
    const float beta = TAU_F - acosf(k - 1.0f);             // T3: the one root of dT/dbeta = 0
    return beta < phi ? deg(beta) : -1.0f;
}

// Algorithm 1.3 (PlanLeg): the faster of "steer phi, drive d" and the best detour.
LegPlan DetourSteer::plan(float phiDeg, float d, const RobotParams& p) const {
    LegPlan direct = DirectPlanner().plan(phiDeg, d, p);
    float k = 0.0f;
    const float betaDeg = optimalDetourAngleDeg(phiDeg, d, p, &k);
    direct.k = k;
    if (betaDeg < 0.0f) return direct;

    const float phi = rad(phiDeg), beta = rad(betaDeg);
    const float s = sinf(beta);
    const float a = d * sinf(beta - phi) / s;               // leg 1: heading unchanged
    const float b = d * sinf(phi) / s;                      // leg 2: heading beta*, ends at the goal
    const float t = a / p.driveMps + p.stopS + betaDeg / p.steerDps + p.settleS + b / p.driveMps;
    if (!(t < direct.timeS)) return direct;                 // the extra stop must pay for itself

```

```
    LegPlan l;
    l.kind = LegPlan::Detour;
    l.a = a;
    l.betaDeg = betaDeg;
    l.b = b;
    l.timeS = t;
    l.k = k;
    return l;
}
```